

Projektuppgift

Fullstack applikation till restaurang

Kevin Maninnerby

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Kevin Maninnerby, kema2501@student.miun.se

Utbildningsprogram: TWEUG, Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: VT, 2026

1 Sammanfattning

I detta projekt beskrivs utvecklingen av en fullstack applikation för en restaurang i premiumsegmentet. Syftet var att skapa en modern applikation där besökare enkelt kan hitta information, läsa menyn, skicka meddelanden och göra bordsreservationer, samtidigt som restaurangens personal ska kunna administrera denna data i ett administratörsgränssnitt.

Projektet inleddes med designarbete i Figma och databasplanering genom ER-diagram. Applikationen utvecklades med en tydlig uppdelning mellan klient- och administratörssidan (frontend) samt serversidan (backend). Applikationens frontend byggdes med plattformen Angular, där flera av Angulars inbyggda funktioner använts för att uppnå applikationens funktionalitet. Backend-servern utvecklades i Express.js och kommunicerar via HTTP-anrop med en MongoDB-databas för lagring av meny, bokningar, och meddelanden. För att skapa en säker applikation implementerades metoder som HTTPS, lösenordshashning med Bcrypt samt behörighetskontroll via JSON Web Tokens (JWT) och skyddade rutter.

Resultatet blev en fullt fungerande webbapplikation som uppfyller samtliga uppställda krav. Feedback från kurskamrater visade att designen upplevs som genomarbetad och anpassad för målgruppen. De största tekniska utmaningar under projektets gång rörde tidsåtgången för det visuella gränssnittet samt hanteringen av tidsomvandling mellan lokal tid och UTC, vilket löstes genom att lagra bokningsdata i strängformat.

Sammanfattningsvis har projektet resulterat i en säker och användarvänlig plattform som underlättar restaurangens dagliga verksamhet.

2 Innehållsförteckning

1	Sammanfattning	ii
2	Innehållsförteckning	iii
3	Introduktion.....	1
4	Teori	2
4.1	Hur fungerar en server och webbtjänster?	2
4.2	Vad är en databas?.....	2
4.3	Vad är webbsäkerhet och dataskydd?	3
5	Metod	5
6	Konstruktion	6
6.1	Design och grafisk profil	6
6.2	Backend.....	6
6.3	Datavalidering och säkerhetsfunktioner	8
6.4	Frontend och Angular	9
7	Resultat	12
8	Slutsatser	13

3 Introduktion

Denna rapport beskriver utvecklingen av fullstack applikation åt Pretty Decent, en fiktiv restaurang på Södermalm i Stockholm. Applikationens primära syfte är att marknadsföra restaurangens verksamhet och den mat som serveras. För att företaget och dess kunder ska kunna använda applikationen effektivt finns det ett flertal problem som behöver lösas.

Det första problemet som ska lösas är att skapa en snygg och välarbetad design så att kunder enkelt kan hitta relevant information om restaurangen och deras besök. Det ska vara möjligt för gäster att läsa menyn, göra bordsbokningar samt skicka meddelanden till restaurangen, via applikationen. Applikationen behöver även vara responsiv, så att användare kan använda applikationen på olika enheter, såsom dator och mobil.

Ett annat stort problem är att restaurangpersonalens själva behöver kunna hantera sitt digitala innehåll utan att behöva teknisk support. Projektet innefattar därför att bygga applikationen som ett Headless CMS. Det innebär att det ska skapas ett administrationsgränssnitt där restaurangen själva ska kunna uppdatera menyn, se över bordsbokningar och läsa inkomna meddelanden.

För att lösa dessa problem i praktiken omfattar projektet att bygga både en baksida (backend) och en framsida (frontend). Applikationens server ska konstrueras med hjälp av Express.js och kunna kommunicera med en databas i MongoDB. För att underlätta denna kommunikation och skapa strukturerade och säkra scheman kommer Mongoose att användas. Applikationen behöver även använda moderna metoder för att skydda den data som lagras i databasen.

På framsidan ska användargränssnittet byggas upp i ramverket Angular. Det konkreta arbetet blir att skapa olika vyerna för restaurangkunder såväl som ett administrationsgränssnitt för personalen samt att konstruera tjänster med Http-Client så att data kan hämtas och skickas till servern.

4 Teori

4.1 Hur fungerar en server och webbtjänster?

En *server* är en dator eller enhet som lagrar, bearbetar och skickar data till andra datorer via ett nätverk. En server fungerar som en samlingspunkt som tillhandahåller tjänster till andra enheter, så kallade klienter [1]. För att servern ska kunna utföra specifika uppgifter behövs en mjukvara som hanterar logiken, till exempel en webbtjänst [4].

En webbtjänst är ett program som körs via ett nätverk och erbjuder funktionalitet eller data till andra applikationer. I stället för att en applikation ansluts direkt till en databas kan en webbtjänst agera som en mellanhand. Webbtjänsten kan sköta intern logik och skicka svar på förfrågningar, oftast i JSON eller XML. Användningen av webbtjänster möjliggör att flera klienter såsom webbplatser, mobilappar och servrar, kan dela samma funktionalitet [2].

För att strukturera hur kommunikationen mellan olika klienter sker används vanligtvis ett API (Application Programming Interface). Ett API är själva gränssnittet eller regelverket som beskriver hur klienter får kommunicera med en webbtjänst. API:et specificerar bland annat vilka adresser som finns, vilken data som kan skickas samt vilket svar som returneras [2].

REST (Representational State Transfer) är en vanlig arkitekturstil för utformning av webbtjänster. Arkitekturen bygger på att systemets data och funktioner betraktas som resurser, som nås via specifika URL:er. För att hämta och manipulera data används HTTP-metoder. Varje metod motsvarar en unik operation. GET används för att hämta data, POST för att skapa ny data, PUT för att uppdatera existerande data och DELETE för att radera data [2].

Express.js är ett ramverk för Node.js, skapat för att effektivisera och förenkla processen att skapa webbapplikationer och API:er. Medan Node.js har grundläggande inbyggda funktioner för att hantera nätverkstrafik, tillhandahåller Express.js färdiga verktyg för att implementera HTTP-metoder, hantera sökvägar (routing) samt bygga ett REST-baserade API:er [3].

4.2 Vad är en databas?

En databas är en samling av information som är relaterade till varandra och lagras i ett datorsystem. Det finns en mängd olika *databasservrar* (DBMS), som är system för att hantera databaser. Dessa databassystem kan delas in i relationsdatabaser och icke-relationsdatabaser. Relationsdatabaser är strikt strukturerade och använder programmeringsspråket *SQL* (Structured Query Language). Icke-relationsdatabaser saknar denna fasta struktur och kan

vara utformade på olika sätt. Eftersom många icke-relationsdatabaser inte använder SQL kallas de ofta för NoSQL-databaser [5].

Vid planeringen av en databas struktur används ofta ER-diagram (Entitets-Relationsdiagram). Dessa diagram används för att modellera och åskådliggöra relationerna mellan olika entiteter i databassystemet. I en relationsdatabas representerar entiteterna vanligtvis tabeller, där varje tabell har specifika attribut som beskriver egenskaper. Relationerna definierar sambandet mellan olika tabeller [7]. ER-diagram kan även användas vid planering av NoSQL-databaser, såsom MongoDB. För NoSQL-databaser representeras dock data annorlunda då traditionella tabellrelationer ofta ersätts av inbäddade dokument eller referenser inom kollektioner [8].

4.3 Vad är webbsäkerhet och dataskydd?

Vid skapande av en webbtjänst är det viktigt att data hanteras på ett säkert sätt, så att inte obehöriga får tillgång till känsliga data eller missbruka applikationens funktionalitet. Det är även viktigt att applikationen är konstruerad på ett sådant sätt att den förebygger dataintrång och missbruk av tjänsten [12, 02:00]. Nedan beskriv några metoder och verktyg för att uppnå detta.

Grundläggande inom systemsäkerhet är hanteringen av miljökonfigurationer. För att skydda känslig information och miljöberoende inställningar, som till exempel API-nycklar, databasanvändare, lösenord, etcetera, kan en *Env-fil* användas. Filen innehåller nyckel-värde-par, där varje rad representerar en variabel och dess värde. Variablernas kan sedan användas i applikationen för att hämta konfigurationsdata. Env-filerna ska inte versionhanteras eller laddas upp till ett online repository, utan konfigureras direkt i publiceringsplattformens inställningar [9].

Medan Env-filer skyddar systemet miljökonfigurationer krävs andra mekanismer för att skydda databasservrar mot externa instruktioner, så kallade SQL-injections. Innan data lagras i en databasserver bör den alltid valideras och kontrolleras. *SQL-injections* innebär ett allvarligt säkerhetshot där angripare injicerar skadlig kod i en SQL-fråga, vilket kan resultera i att data manipuleras, förstörs eller läcks. Ett effektivt sätt att skydda en applikation mot SQL-injections är att använda parametriserade SQL-frågor. Det innebär att användares input strikt behandlas som data och separeras från den körbara koden, vilken förhindrar att den skadliga koden körs i databasen [6].

Säkerheten i en databas är i sin tur beroende av hur åtkomsten till systemets resurser kontrolleras. Inom REST-API används begreppen *autentisering* och *auktorisering* för att säkerställa att endast auktoriserade och autentiserade användare eller system får åtkomst till ett API och dess resurser. För att användaren eller klienten ska autentiseras behöver den tillhandahålla

autentiseringsuppgifter, oftast användarnamn och lösenord, API-nyckel eller någon annan typ av identifieringsinformation. Auktoriseringen avgör vilka resurser eller handlingar användaren eller systemet har rätt att nå eller utföra. I praktiken används ofta tokens för att både hantera autentisering och auktorisering [10].

En etablerad standard för token-baserad behörighetskontroll är JWT (JSON Web Token). Dessa tokens används för att verifiera om en användare har behörighet till en resurs. JWT-tokens består av tre delar, header, payload och signature [11].

Eftersom autentiseringstokens och användaruppgifter ständigt överförs i ett klient-server-system, behövs ett metoder både för att skydda data som transporteras och lagras. Detta eftersom känslig information alltid riskerar att exponeras om den hanteras i klartext. För att skydda informationen kan två olika metoder användas, *kryptering* och *hashning* [12, 40:00].

Kryptering är en tvåvägskryptering som omvandlar data till en oläsbar textsträng med hjälp av en algoritm och en nyckel, som krävs för att kunna återställa informationen. Denna metod används främst för att skydda data, så att data aldrig skickas över nätverk i klartext. Vanligtvis används protokollet HTTPS för att kryptera information mellan klient, server och databas. Genom att använda HTTPS förhindras obehöriga att kunna avlyssna och läsa data eller autentiseringsuppgifter som skickas till och från webbtjänstens API [12, 40:00].

För data som ska lagras i databasen, såsom lösenord och annan känslig information används i stället hashning. Till skillnad från kryptering är hashning en envägsprocess, vilket innebär att processen inte går att reversera. Databasen lagrar det hashed lösenordet som en sträng, och vid inloggning hashas användarens angivna inloggningsuppgifter på samma sätt för att se om de båda resultaten matchar [12, 40:00].

Utöver kryptering av HTTP-anrop och behörighetskontroll innefattar webbsäkerhet även regleringen av kommunikation mellan domäner. CORS (Cross-Origin Resource Sharing) är en webstandard som styr säkerhetsprincipen gällande Cross-Origin-anrop, d.v.s. anrop från en annan domän än den ursprungliga laddade webbsidan. CORS är designat för att möjliggöra säker kommunikation mellan webbläsare och servrar från olika domäner [13].

5 Metod

Utvecklingsprocessen inleddes i designverktyget Figma, där en skiss över webbplatsens layout och struktur skapades. Detta gjordes för att planera vilken information som skulle visas för användare. Att ha en färdig design underlättade planeringen av vilken data som behövdes lagras i databasen. Det sparade även tid vid konstruktionen av webbplatsen då designbeslut redan var tagna.

Vidare skapades en planering över vilken data som behövde lagras. Ett ER-diagram ritades upp för att planera databasens schema. Samtidigt planerades vilka API-endpoints som behövdes skapas för servern samt vilka av dessa som behövdes skyddas med inloggning. MongoDB valdes som databas eftersom databasen erbjuder en flexibel NoSQL-struktur som förenklar lagringen av JSON-liknande dataobjekt. Eftersom applikationen inte krävde komplexa relationer eller joins mellan databaser ansågs det vara ett lämpligt alternativ.

Applikationens backend-server byggdes i Express.js. För att hantera databasen och säkerheten användes flera NPM-paket. Mongoose användes för att hantera databasen i MongoDB. Bcrypt användes för att hashning av lösenord och JSON Web Tokens (JWT) användes för att skapa tokens till de skyddade rutterna. Dessutom lades CORS till för att tillåta kommunikation mellan applikationens backend och frontend.

Applikationens frontend utvecklades i Angular. Angular är en modulär SPA-plattform (Single Page Application) där applikationens delar skapas som fristående komponenter. I projektet skapades delar såsom components, services och guards. Därtill användes Http-Client för HTTP-anrop samt Signals för reaktiv data.

Under projektet utveckling versionshanterades projektet med hjälp av Git. Vid publicering laddades projektet upp till Github. Applikationens backend och frontend distribuerades via Render, medan applikationens databas använder sig av MongoDBs server Atlas.

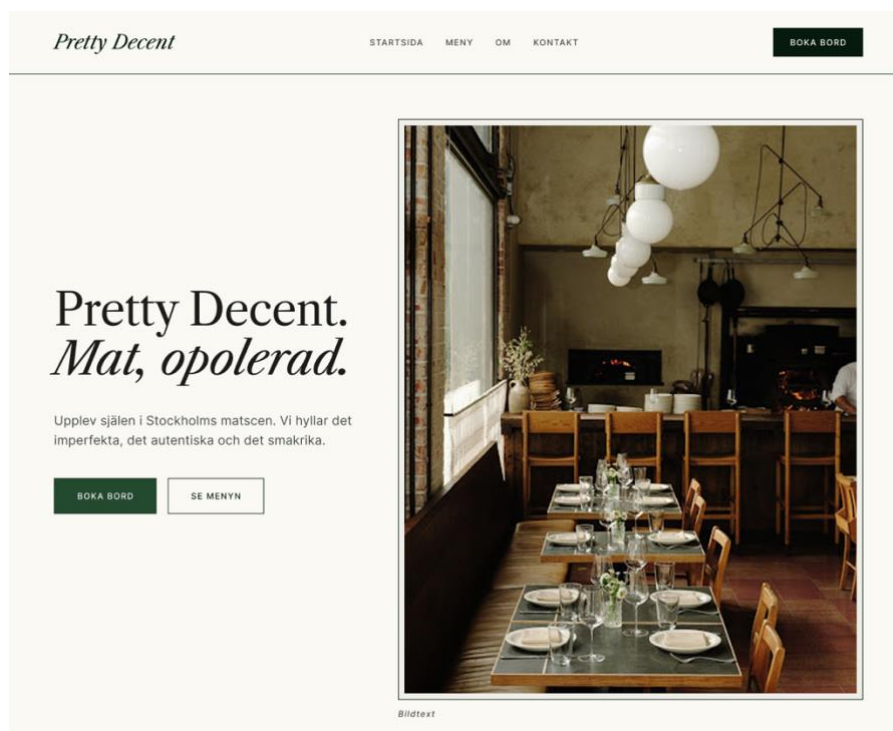
I utvecklingen av webbtjänsten användes API-klienten Bruno som verktyg för att testa systemets olika endpoints innan de integrerades i frontend. Användningen av Bruno gjorde också att provkörningen kunde kringgå eventuell CORS-problematik.

I utvecklingsprocessen användes Google Chrome DevTools för att testa applikationens responsivitet. Eftersom frontend-delen är byggd med Angular och index.html enbart är ett tomt skal, validerades inte applikationen.

6 Konstruktion

6.1 Design och grafisk profil

Utvecklingen av applikationen inleddes med att utarbeta en grafisk profil för användargränssnittet (frontend). För att effektivisera processen valdes två nyckelsidor ut: startsidan och menyn. Dessa sidor ansågs tillräckliga för att lägga grunden för webbplatsens visuella identitet och för att ge en överblick över vilken data som behövdes lagras i databasen. För startsidan skapade även en mobilvy för att modellera hur webbplatsen anpassas för mindre skärmstorlekar (se Figur 1 samt bilaga A-C).

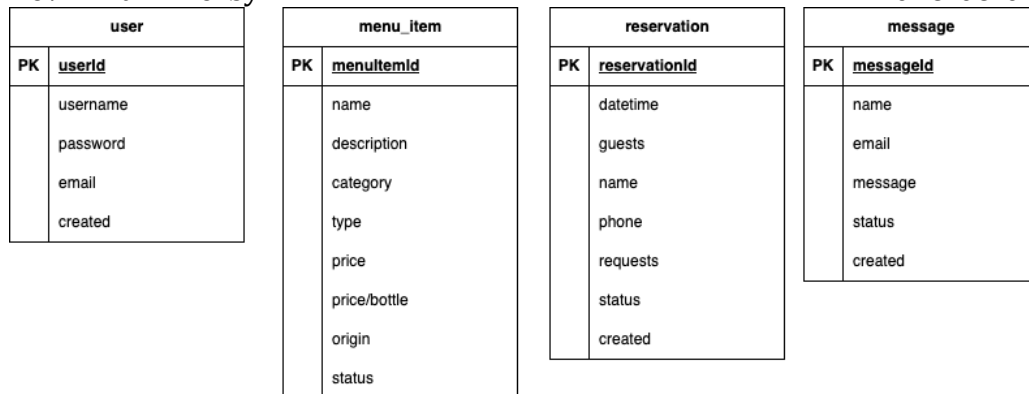


Figur 1- Grafisk profil startsidea

Tidigt i planeringsprocessen bestämdes restaurangens motto till "celebrate the imperfect", vilket medvetet återspeglades i designen. Detta genomfördes genom att kombinera sans-serif- och serif-typsnitt, samt att använda relativt oredigerade bilder. Genom att använda en avskalad design med få färger, lyfts text och bild fram som webbplatsens centrala element.

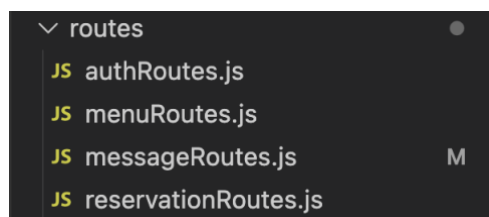
6.2 Backend

Applikationens databas och backend-server planerades genom att rita upp ett enkelt ER-diagram på ett papper, samt skriva ut REST-webbtjänstens endpoints. För tydlighetens skull har ER-diagramet renskrivits, se Figur 2. Notera att användarnamn har tagits bort från user, samt att price/bottle och origin tagits bort från menu_item.



Figur 2 - ER-diagram

Backend-server konstruerades i Node.js med ramverket Express. Under utvecklingen av servern användes npm-paketet Nodemon för att förenkla testningen av servern. För att hantera känsliga anslutningsuppgifter till databasen utan att inkludera dem i projektets källkod användes npm-paketet Dotenv. Servers struktur organiserades med hjälp av npm-paketet Router, vilket användes för att dela upp applikationens olika endpoints i separata filer och mappar för att skapa en tydlig struktur (se Figur 3).



Figur 3- rutter

För att upprätta en anslutning till MongoDB och definiera scheman användes Mongoose. Eftersom MongoDB är en NoSQL-databas finns ingen sårbarhet för traditionella SQL-injektioner, men databasen kan fortfarande skadas av andra typer av skadliga injektioner. Detta förebyggdes genom att använda Mongoose strikta schema för all data som lagras i databasen (se Figur 4).



```
// Messages schema
const messageSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    match: [/^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/ ,
    "Please fill a valid email address"]
  },
  title: {
    type: String,
    required: true
  },
  message: {
    type: String,
    required: true,
    min: [10]
  },
  status: {
    type: String,
    enum: ["new", "pending", "answered"],
    default: "new"
  },
  created: {
    type: Date,
    default: Date.now
  }
});
```

Figur 4 - Mongoose schema

6.3 Datavalidering och säkerhetsfunktioner

Mongoose-scheman användes även för att validera att inkommande data från användaren uppfyller systemets krav. I schemat för meddelandefunktionen (Figur 4) kontrolleras exempelvis att e-postadressen följer en korrekt struktur (till exempel name@gmail.com) samt att själva meddelandet består av minst 10 tecken. I bokningsfunktionen används logik i schemat för att säkerställa att bokningsdatumet ligger framåt i tiden.

När ett nytt användarkonto skapas samverkar Mongoose-schemat med npm-paketet Bcrypt för att hasha lösenordet innan det lagras i databasen (se Figur 5). När användaren sen ska logga in kontrolleras den lagrade hasningen mot en hashing av det angivna lösenordet, stämmer lösenorden överens beviljas användaren åtkomst till webbplatsens administrationsgränssnitt. I samband med att inloggningen lyckas generas också en JSON Web Token (JWT) med hjälp av npm-paketet med samma namn. Token returneras i API-svaret och kan sedan lagras och användas av applikationens frontend. Genom att använda token kan användaren göra anrop till skyddade rutter (routes) utan att behöva identifiera sig på nytt.

```
// Hash password
userSchema.pre("save", async function() {
  try {
    if(this.isNew || this.isModified("password")) {
      const hashedPassword = await
        bcrypt.hash(this.password, 10);
      this.password = hashedPassword;
    }
  } catch(error) {
    throw error;
  }
});
```

Figur 5- hashning

De rutter som behövde skyddas i projektet var hantering av menyn (skapa, ändra och ta bort rätter) samt hantering av bordsbokningar och meddelanden. För att skydda dessa endpoints skapades ett middleware vid namn `auth_tokens`. Denna middleware kontrollerar att HTTP-anropet innehåller en giltig JWT-token innan åtkomst tillåts (se Figur 6).

```
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (token == null) {
    return res.status(401).json({ message: "Not authorized for this route - token missing!" });
  }

  jwt.verify(token, process.env.JWT_SECRET_KEY, (err, email) => {
    if(err) return res.status(403).json({ message: "Unvalid JWT" });
    req.email = email;
    next();
  });
}
```

Figur 6 - Middleware

Slutligen installerades paketet CORS på servern för att tillåta att applikationens frontend kan utföra http-anrop från en annan domän än servers.

6.4 Frontend och Angular

Webbplatsens frontend konstruerades med ramverket Angular. Användargränssnittet delades upp i komponenter baserat på webbplatsens olika sidor: about, admin, booking, contact, homepage, login, menu samt not-found. Utöver dessa skapades även komponenterna header och footer, vilka placerades i en separat layout-mapp för att hantera webbplatsens övergripande struktur.

För att hantera HTTP-kommunikationen med backend-servern skapades fyra Angular-tjänster (services): auth-service, booking-service, contact-service och menu-service. I dessa tjänster användes HTTP-client för att göra HTTP-anrop till backend-servern. För att säkerställa att data som skickas och tas emot via API-förfrågningarna strukturerades i rätt format skapades TypeScript interfaces med hjälp av interface funktionen i Angular, se Figur 7.

```
export interface MenuItem {  
  name: string;  
  description: string;  
  type: MenuItemType;  
  category: string;  
  price: number;  
}
```

Figur 7 - Interface menuItem

Genom att använda Angular Guards behöves inte administrationsgränssnittet inte byggas som en extern applikation utan kunde smidigt integreras direkt i webbplatsen. Den skapade guarden kontrollerar om det finns en sparad JWT-token i webbläsarens LocalStorage innan användaren ges tillgång till admin-sidan, se Figur 8. Om en token saknas omdirigeras användaren till inloggningssidan. Av tidsmässiga skäl prioriterades funktionalitet för att hålla koll på tokenens giltighetstid bort i frontend. Skulle en användare ha en utgången token blockeras anropen fortfarande av serverns middleware, men användare måste i detta fall logga ut och logga in igen manuellt för att få åtkomst.

```
export const authGuardGuard: CanActivateFn = (route, state) =>  
{  
  const authService = inject(AuthService);  
  const router = inject(Router);  
  
  if(authService.isLoggedIn()) {  
    return true;  
  }  
  router.navigate(['/login']);  
  return false;  
};
```

Figur 8 - Guard

Applikationen använder sig av moderna metoder som till exempel Signaler (Signals). Genom att använda signaler kan webbplatsen innehåll enkelt laddas om dynamiskt utan att hela sidan behöver laddas om. Signalerna är reaktiva och känner av när en ändring görs till dem. Detta används till exempel vid utskrift av felmeddelanden samt redigering av menyartiklar. I administratörsgränssnittet använder sig alla HTTP-anrop av samma meddelanderuta, för att minska risken att användaren missar återkoppling från tjänsten.

Vid redigering av menyartiklar används även så kallad databinding, där formulärsdata kopplas direkt till variabler. Detta underlättar redigeringen av menyartiklarna avsevärt, där data kan läsas in och redigeras direkt i inputfält, se Figur 9.

A screenshot of a code editor with a dark background. It shows an Angular template snippet for a form field. The code is as follows:

```
<label> Namn:
  <input type="text" [(ngModel)]="reservation.name" name="name"
    class="edit-field">
</label>
```

Figur 9- Databinding i administratörsvy

Utöver att redigera menyartiklar kan administratören se och redigera meddelanden och bordsbokningar i administratörsvy. Även dessa använder sig av databinding där deras status enkelt kan ändras med hjälp av en rullista. Bordsreservationerna visas även i en fallandra ordning, utifrån datum. Bordsreservationer som redan har varit filtreras dessutom bort. Denna filtrering görs efter midnatt varje dag eftersom restaurangen förmodligen vill kunna se bordsbokningarna under hela kvällen. Skulle till exempel gästerna vara lite sena så försvinner inte bordsbokningen.

7 Resultat

I detta avsnitt presenteras projektets resultat utifrån de problemformuleringar och krav som ställdes på webbapplikationen under planeringen.

Applikationen har utvecklats med fokus på användbarhet och en tydlig struktur, så att det ska vara enkelt för besökare att hitta information om restaurangen, läsa menyn, göra bordsbokningar samt skicka meddelanden. Gränssnittet har även utformats med en responsiv design, vilket innebär att det automatiskt anpassar sig för att fungera väl på olika skärmstorlekar. Responsen från kurskamrater gällande applikationens design har varit positiv, vilket tyder på att utseendet upplevs som genomarbetat och passande för en restaurang i premiumsegmentet.

För att restaurangen ska kunna hantera sin dagliga verksamhet har en lösenordskyddad administratörsvy skapats. Genom detta gränssnitt har behöriga användare möjlighet att uppdatera och förändra restaurangens meny, samt övervaka och hantera inkommande bordsbokningar och meddelanden från kunder.

Systemets arkitektur är uppbyggd med en tydlig uppdelning mellan klientsida och serversida. Applikationens framsida (frontend) är utvecklat i plattformen Angular, där applikationens olika delar har strukturerats som fristående komponenter. All data som rör meny, reservationer och meddelanden lagras i en MongoDB-databas. Kommunikationen mellan systemets olika delar samt ändringar av databasens innehåll sker via http-förfrågningar via applikationens tillhörande server och webbtjänst.

För att säkerställa att applikationen är säker och att känslig data skyddas har flera säkerhetsmetoder använts. All datatrafik mellan klienten och servern krypteras med hjälp av HTTPS. Applikationen använder sig dessutom av JWT-tokens för att kontrollera behörighet och säkerställa att endast auktoriserade användare har tillgång till administratörsvyn. Slutligen skyddas känsliga uppgifter i databasen genom hasning, vilket innebär att inga lösenord lagras i klartext.

8 Slutsatser

Genomförandet av projektet har varit både givande och utvecklande. Redan tidigt i projektet skapades en tydlig plan för applikationens struktur och funktionalitet, vilket gjorde utvecklingsprocessen både strukturerad och effektiv. Även om problem uppstått under arbetets gång, har de aldrig varit av sådan omfattning att projektet stannat av helt.

Den främsta utmaningen i projektet har varit att balansera tidsåtgången för applikationens användargränssnitt. Applikationens designmall och utformningen av komponenterna tog väldigt lång tid. I efterhand kan konstateras att de visuella aspekterna av applikationen tilläts ta för mycket, men eftersom applikationens logik fungerar väl bedöms ändå slutresultatet som lyckat.

En bidragande faktor till att inga större problem uppstått i utvecklingen av applikationen kan vara en förbättrad metodik gällande felhantering. Genom att implementera tydliga och felmeddelanden på serversidan underlättades felhanteringen avsevärt. Genom tydliga felmeddelanden har problemen oftast kunnat lokaliseras och åtgärdas relativt snabbt.

Projektet har bidragit till en djupare förståelse för hur en backend-server kan struktureras med hjälp av Express. Ett lyckat moment i projektet var skapandet av en återanvändbar middleware för samtliga skyddade rutter.

I tidigare projekt har hanteringen av Cross-Origin Resource Sharing (CORS) inneburit stora problem, men under detta projekt uppstod inte dessa problem i samma utsträckning. CORS-problematik uppstod enbart i samband med att backend-delen publicerades till Render medan frontend-delen fortfarande kördes lokalt. Detta löstes dock snabbt genom att publicera applikationens frontend på samma plattform.

Den största tekniska motgången under projektet rörde hanteringen av datum och tider. I projektet användes HTML-formuläret `datetime` i administratörsgränssnittet, vilket baseras på lokal tid, medan MongoDB lagrar tidsdata i UTC-format. Genom att försökt skapa funktionalitet i applikationens frontend för omvandling mellan lokaltid och UTC-format togs beslutet att i stället lagra all tidsdata kopplat till bordsbokning i strängformat. Tillvägagångssättet valdes för att kringgå komplexiteten som tidsomvandlingen medförde för att säkerställa att bordsbokningen fungerade på säkert och stabilt sätt.

Källförteckning

- [1] One.com. Vad är en server? En nybörjarguide till servrar och deras funktioner [internet]. Köpenhamn: One.com [citerad: 2026-06-01]. Hämtad från: <https://www.one.com/sv-se/vps/vad-ar-en-server/>
- [2] Mittuniversitet. Moment 2 – översikt [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/oversikt-moment-2?module_item_id=156946
- [3] Mittuniversitet. Express och EJS [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/express-och-ejs?module_item_id=206200
- [4] Mdn. What is a web server? [internet]. San Francisco: MDN [citerad: 2026-06-01]. Hämtad från: https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server
- [5] Microsoft. Vad är databaser? [internet]. [citerad: 2026-06-01]. Hämtad från: <https://azure.microsoft.com/sv-se/resources/cloud-computing-dictionary/what-are-databases>
- [6] Mittuniversitet. SQL-injections och parameteriserade SQL-frågor [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/sql-injections-och-parameteriserade-sql-fragor?module_item_id=206208
- [7] Mittuniversitet. Er-diagram [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/er-diagram?module_item_id=206203
- [8] GeeksforGeeks. Equivalent of ERD for MongoDB. Noida: GeeksforGeeks [uppdaterad: 2025-07-23, citerad 2026-06-01]. Hämtad från: <https://www.geeksforgeeks.org/mongodb/equivalent-of-erd-for-mongodb/>
- [9] Mittuniversitet. Env-filer [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/env-filer?module_item_id=206206
- [10] Mittuniversitet. Autentisering och auktorisering [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från:

https://canvas.miun.se/courses/8214/pages/autentisering-och-auktorisering?module_item_id=206210

- [11] Mittuniversitet. JWT- JSON Web Tokens [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/jwt-json-web-tokens?module_item_id=206212
- [12] Mittuniversitetet. Autentisering och säkerhet [video]. 2026-04-29 [citerad: 2026-06-01]. Hämtad från: https://play.miun.se/media/Backend-baserad+webbutveckling/0_lf2tuki
- [13] Mittuniversitet. Cross-Origin Resource Sharing [internet]. Sundsvall: Mittuniversitetet [citerad: 2026-06-01]. Hämtad från: https://canvas.miun.se/courses/8214/pages/cors-cross-origin-resource-sharing?module_item_id=220877

9 Bilagor

9.1 Bilaga A - Startside

Pretty Decent

STARTSIDA MENY OM KONTAKT

BOKA BORD

Pretty Decent. Mat, opolerad.

Upplev själen i Stockholms matscen. Vi hyllar det imperfekta, det autentiska och det smärrika.

BOKA BORD SE MENYN



Bildtext

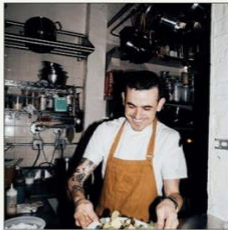
Vår vibe

— STOCKHOLM, SÖDERMALM



01. ATMOSFÄREN

Linneservetter, levande ljus och ett kurerat utval av goda viner.



02. HANTVERKET



03. GEMENSKAPEN

Intellektuellt nyfiken

Vi är inte intresserade av perfektion. Vi bryr oss om ingrediensernas berättelse, receptens historier och konversationerna vid bordet. Vårt kök utvecklas konstant, drivna av säsongens varor och inspirationen från närområdet.

Eftersom inspirationen flödar finns alltid rum för överraskningar. Fråga din servitör om kvällens specialrätt eller lita på våra kockar och dyk in i vår väl utvalda [meny](#).

Pretty Decent

BESÖK

Södermalm, Stockholm

Tisdag — Lördag

17:00 — Sent

KOM I KONTAKT

Instagram

Facebook

Tiktok

POLICY

© 2024 Pretty Decent Stockholm.

All Rights Reserved.

9.2 Bilaga B – Mobilsida



Pretty Decent

STARTSIDA MENY OM KONTAKT

BOKA BORD

TISDAG — LÖRDAG

Dagens meny

STOCKHOLM, SKÖDERMALM

Förrätter

Swedish Oyster

45 / st

Shallot mignonette, fermented chili oil, lemon.

Nduja & Honey Toast

115

Spicy calabrian pork spread, wildflower honey, toasted sourdough.

Salt-baked Beets

135

Goat cheese curd, toasted hazelnuts, elderflower vinaigrette.

Huvudrätter

Pan-seared Cod

320

Brown butter dashi, charred leeks, kohlrabi ribbons.

Slow-cooked Lamb Shank

345

Creamy polenta, gremolata, red wine reduction.

Mushroom Risotto

285

Wild forest mushrooms, 24-month parmesan, truffle essence.





Efterrätter

Burnt Butter Tart

95

Sea salt, mascarpone cream.

Dark Chocolate Ganache

110

Olive oil, Maldon salt, sourdough crisp.

VIN

2022 Pet Nat, Loire Valley	145 / 720
2021 Orange Wine, Slovenia	160 / 800
2020 Pinot Noir, Burgenland	175 / 870

DRINKAR

Negroni Spumante	155
Swedish Martini (Dill & Rye)	165
Burnt Lemon Spritz	145

Pretty Decent

BERÄTTELSE PÅ TALLRIKEN. SAMTAL RUNT BORDET.

BESÖK

Södermalm, Stockholm

Tisdag — Lördag

17:00 — Sent

KOM I KONTAKT

Instagram

Facebook

TikTok

POLICY

© 2024 Pretty Decent Stockholm.

All Rights Reserved.

1